

50 Programming Questions: File I/O, Message Queues, IPC

File Operations in C

- Q1. Write a C program to open a file and read its content.
- Q2. Write to a text file using ``fwrite()`` function.
- Q3. Use ``fseek()`` to jump to the middle of a file and print data.
- Q4. Check if file exists before opening.
- Q5. Count number of lines in a given file.
- Q6. Copy contents of one file to another.
- Q7. Difference between text and binary file handling.
- Q8. How to append data to an existing file.
- Q9. Use ``fscanf()`` to read formatted input from file.
- Q10. Write a program to reverse the content of a file.

POSIX Message Queues

- Q11. Create and open a message queue using ``mq_open()``.
- Q12. Send a message using ``mq_send()`` in C.
- Q13. Receive a message using ``mq_receive()``.
- Q14. Set attributes of a message queue.
- Q15. Write a program to demonstrate blocking receive.
- Q16. Non-blocking send and receive with ``O_NONBLOCK``.
- Q17. Delete a message queue with ``mq_unlink()``.
- Q18. How to handle message queue full condition?
- Q19. Monitor a message queue using ``select()`` or ``poll()``.
- Q20. Create a producer and consumer using POSIX MQ.

System V Message Queues

50 Programming Questions: File I/O, Message Queues, IPC

- Q21. Create message queue with ``msgget()``.
- Q22. Send and receive using ``msgsnd()`` and ``msgrcv()``.
- Q23. Write a program to exchange messages between two processes.
- Q24. Remove message queue using ``msgctl()``.
- Q25. Check for message queue limits.
- Q26. How to send structured messages with System V MQ?
- Q27. Handle `IPC_NOWAIT` flag in receive.
- Q28. Differentiate System V vs POSIX queues.
- Q29. Blocking behavior in System V MQ.
- Q30. How to handle permission errors in MQ?

Other IPC Mechanisms (Pipes, Shared Memory, Semaphores)

- Q31. Write a program to communicate using unnamed pipes.
- Q32. Use ``fork()`` and pipe for parent-child communication.
- Q33. Create a named pipe (FIFO) and use it for IPC.
- Q34. Create and attach shared memory segment using ``shmget()`` and ``shmat()``.
- Q35. Write and read data using shared memory.
- Q36. Detach and delete shared memory using ``shmdt()`` and ``shmctl()``.
- Q37. Synchronize access to shared memory using semaphores.
- Q38. Implement mutual exclusion using ``semop()`` operations.
- Q39. Compare pipes vs message queues vs shared memory.
- Q40. What are the use-cases for each IPC mechanism?